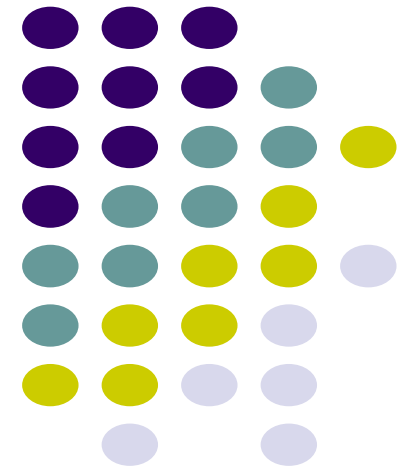


UNIT 4. PROCESSOR



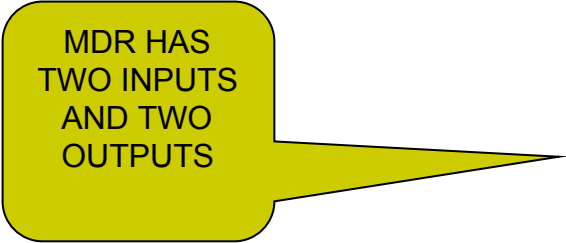
Some Fundamental Concepts

- Processor fetches one instruction at a time and perform the operation specified.
- Instructions are fetched from successive memory locations until a branch or a jump instruction is encountered.
- Processor keeps track of the address of the memory location containing the next instruction to be fetched using Program Counter (PC).
- Instruction Register (IR)
- Suppose each instruction comprises 4 bytes, and that is stored in one memory word. To execute an instruction, the process will do following

SOME FUNDAMENTAL CONCEPTS

- To execute an instruction, processor has to perform following 3 steps:
 - 1) Fetch contents of memory-location pointed to by PC. Content of this location is an instruction to be executed. The instructions are loaded into IR, Symbolically, this operation is written as:
$$IR \leftarrow [PC]$$
 - 2) Increment PC by 4.
$$PC \leftarrow [PC] + 4$$
 - 3) Carry out the actions specified by instruction (in the IR).
- The first 2 steps are referred to as **Fetch Phase**.
Step 3 is referred to as **Execution Phase**.
- The operation specified by an instruction can be carried out by performing one or more of the following actions:
 - 1) Read the contents of a given memory-location and load them into a register.
 - 2) Read data from one or more registers.
 - 3) Perform an arithmetic or logic operation and place the result into a register.
 - 4) Store data from a register into a given memory-location.

Processor Organization



MDR HAS
TWO INPUTS
AND TWO
OUTPUTS

- An instruction is executed by performing one or more of the following operations:
 - 1) Transfer a word of data from one register to another or to the ALU.
 - 2) Perform arithmetic or a logic operation and store the result in a register.
 - 3) Fetch the contents of a given memory-location and load them into a register.
 - 4) Store a word of data from a register into a given memory-location.

Register Transfer

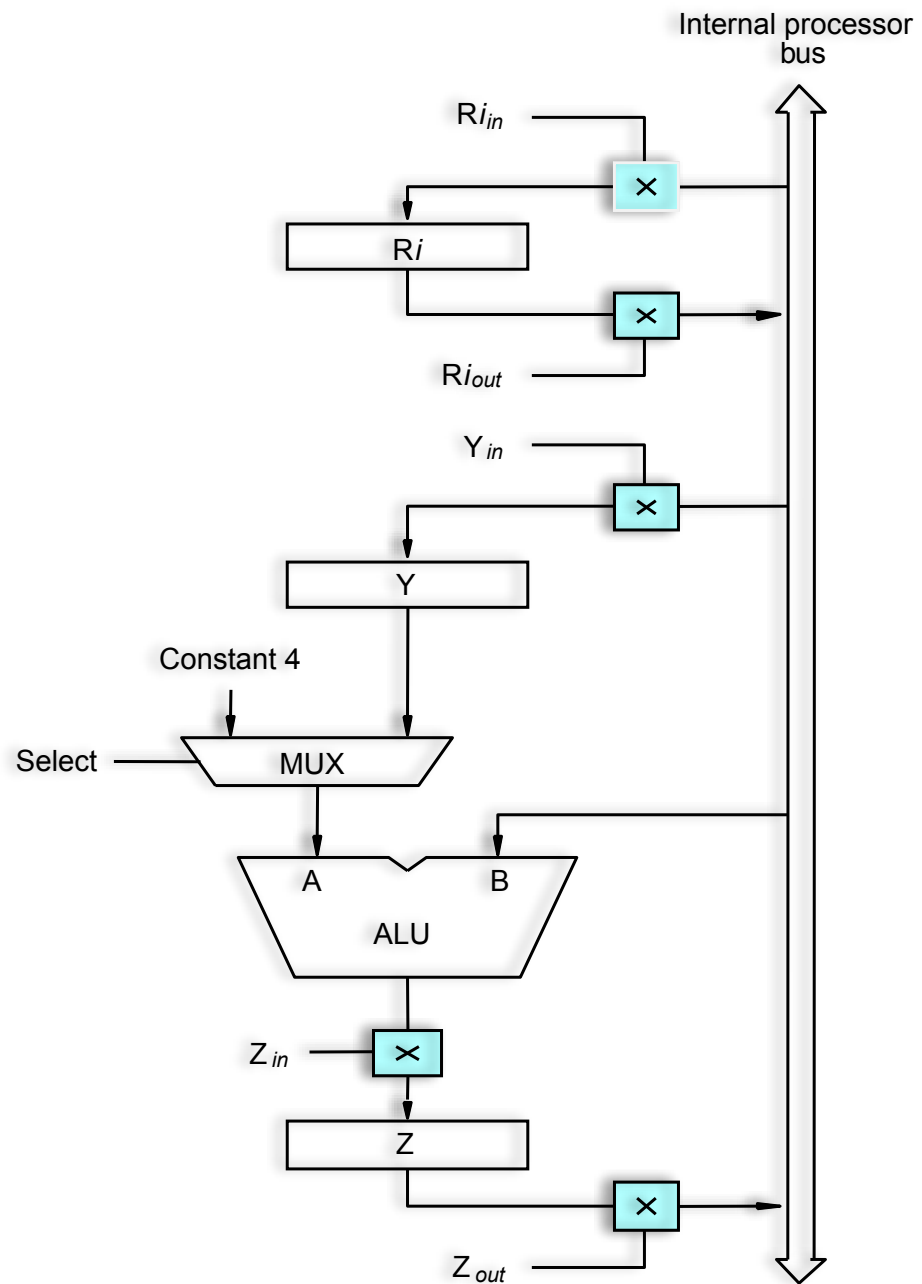


Figure 7.2. Input and output gating for the registers in Figure 7.1.

PERFORMING AN ARITHMETIC OR LOGIC OPERATION

- The ALU performs arithmetic operations on the 2 operands applied to its A and B inputs.
- One of the operands is output of MUX;
And, the other operand is obtained directly from processor-bus.
- The result (produced by the ALU) is stored temporarily in register Z.
- The sequence of operations for $[R3] \leftarrow [R1] + [R2]$ is as follows:
 - 1) $R1_{out}, Y_{in}$
 - 2) $R2_{out}, \text{SelectY, Add, } Z_{in}$
 - 3) $Z_{out}, R3_{in}$
- Instruction execution proceeds as follows:
 - Step 1 --> Contents from register R1 are loaded into register Y.
 - Step2 --> Contents from Y and from register R2 are applied to the A and B inputs of ALU;
Addition is performed &
Result is stored in the Z register.
 - Step 3 --> The contents of Z register is stored in the R3 register.
- The signals are activated for the duration of the clock cycle corresponding to that step. All other signals are inactive.

FETCHING A WORD FROM MEMORY

- To fetch instruction/data from memory, processor transfers required address to MAR.
At the same time, processor issues Read signal on control-lines of memory-bus.
- When requested-data are received from memory, they are stored in MDR. From MDR, they are transferred to other registers.
- The response time of each memory access varies (based on cache miss, memory-mapped I/O). To accommodate this, MFC is used. (MFC $\rightarrow$ Memory Function Completed).
- MFC is a signal sent from addressed-device to the processor. MFC informs the processor that the requested operation has been completed by addressed-device.
- Consider the instruction Move (R1),R2. The sequence of steps is (

Move (R1), R2

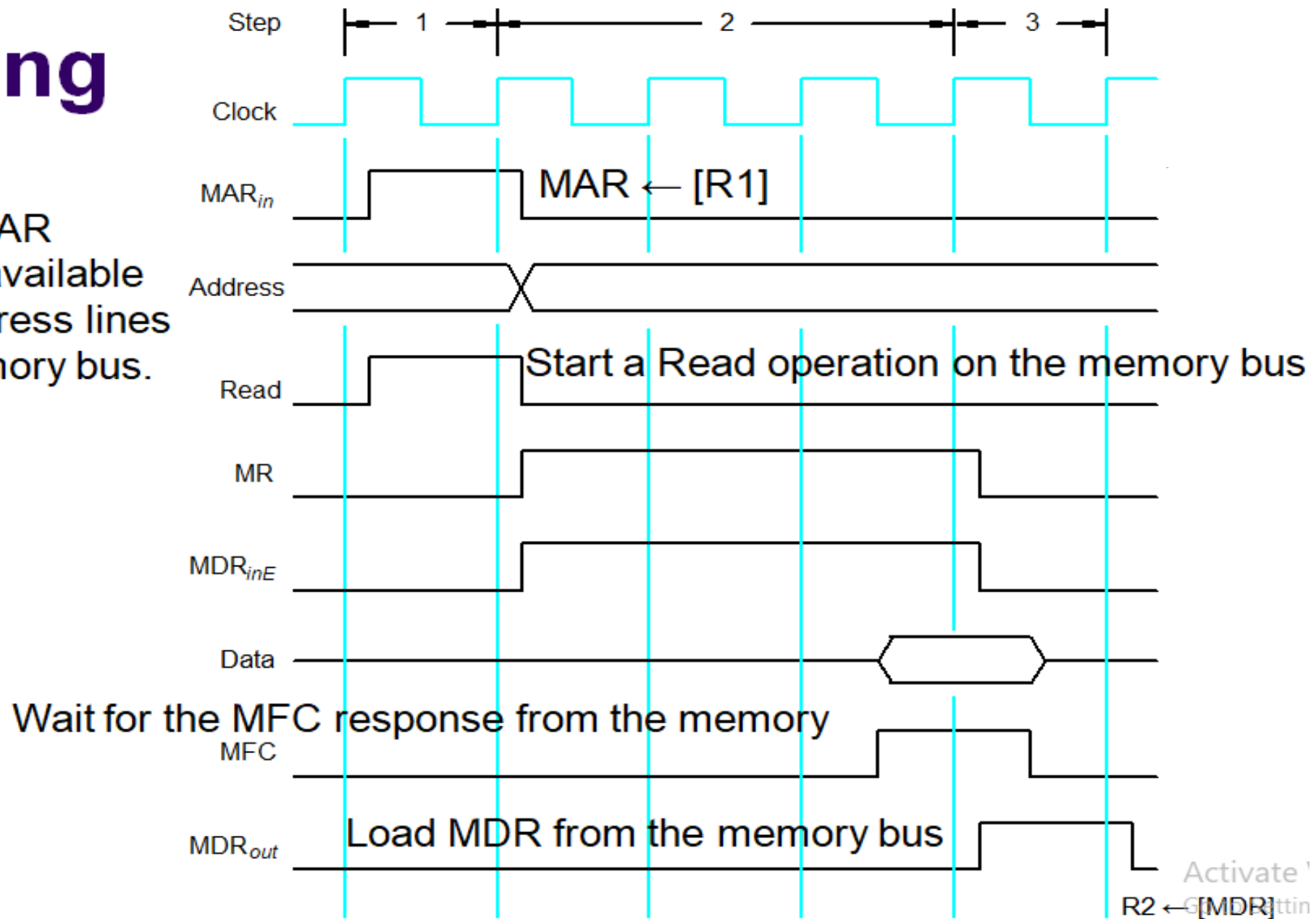
1. MAR $\leftarrow$ [R1]
2. Start a Read operation on the memory bus
3. Wait for the MFC response from the memory
4. Load MDR from the memory bus
5. R2 $\leftarrow$ [MDR]

- 1) R1_{out}, MAR_{in}, Read ;desired address is loaded into MAR & Read command is issued.
- 2) MDR_{inE}, WMFC ;load MDR from memory-bus & Wait for MFC response from memory.
- 3) MDR_{out}, R2_{in} ;load R2 from MDR.

where WMFC=control-signal that causes processor's control.
circuitry to wait for arrival of MFC signal.

Timing

Assume MAR is always available on the address lines of the memory bus.



Storing a Word in Memory

- Consider the instruction *Move R2,(R1)*. This requires the following sequence:

- 1) $R1_{out} \rightarrow MAR_{in}$;desired address is loaded into MAR.
- 2) $R2_{out} \rightarrow MDR_{in}$ Write ;data to be written are loaded into MDR & Write command is issued.
- 3) $MDR_{out} \rightarrow WMFC$;load data into memory-location pointed by R1 from MDR.

- Writing the word into a memory location follows the desired address is loaded into MAR. Then, the data to be written are loaded in to MDR, and a Write command is issued.

EXECUTION OF A COMPLETE INSTRUCTION

- Consider the instruction *Add (R3),R1* which adds the contents of a memory-location pointed by R3 to register R1. Executing this instruction requires the following actions:
 - 1) Fetch the instruction.
 - 2) Fetch the first operand.
 - 3) Perform the addition &
 - 4) Load the result into R1.

Add (R3), R1

Step	Action
1	PC _{out} , MAR _{in} , Read, Select4,Add, Z _{in}
2	Z _{out} , PC _{in} , Y _{in} , WMFC
3	MDR _{out} , IR _{in}
4	R3 _{out} , MAR _{in} , Read
5	R1 _{out} , Y _{in} , WMFC
6	MDR _{out} , SelectY,Add, Z _{in}
7	Z _{out} , R1 _{in} , End

Figure 7.6. Control sequencefor execution of the instruction Add (R3),R1.

Execution of Branch Instructions

Control sequence for an unconditional branch instruction is as follows:

- 1) PCout, MARin, Read, Select4, Add, Zin
- 2) Zout, PCin, Yin, WMFC
- 3) MDRout, IRin
- 4) Offset-field-of-IRout, Add, Zin
- 5) Zout, PCin, End

Execution of branch instruction contd..

The processing starts, as usual, the fetch phase ends in step3.

- In step 4, the offset-value is extracted from IR by instruction-decoding circuit.
- Since the updated value of PC is already available in register Y, the offset X is gated onto the bus, and an addition operation is performed.
- In step 5, the result, which is the branch-address, is loaded into the PC.
- The offset X used in a branch instruction is usually the difference between the branch target-address and the address immediately following the branch instruction. (For example, if the branch instruction is at location 1000 and branch target-address is 1200, then the value of X must be 196, since the PC will be containing the address 1004 after fetching the instruction at location 1000).
- In case of conditional branch, we need to check the status of the condition-codes before loading a new value into the PC.

e.g.: Offset-field-of-IRout, Add, Zin, If N=0 then End

If N=0, processor returns to step 1 immediately after step 4. If N=1, step 5 is performed to load a new value into PC.

Multiple-Bus Organization



Multiple bus organization

- In case of single bus organization only one data item can be transferred over the bus in a clock cycle.
- To reduce number of steps needed, most commercial processors provide multiple internal paths that enable several transfers to take place in parallel.
- In three-bus structure all general purpose registers are combined into a single block called the register file.

- The **register file** is said to have **three** ports
 - Two output-ports allow the contents of 2 different registers to be simultaneously placed on buses A & B.
 - Third input-port allows data on bus C to be loaded into a third register during the same clock-cycle.
- Buses A and B are used to transfer source-operands to A & B inputs of ALU during arithmetic or logic operations.
- Result is transferred to destination over bus C.
- Incrementer-unit is used to increment PC by 4.

Multiple-Bus Organization

- Add R4, R5, R6

Step	Action
1	P _{out} , R=B MA _i , Read IncP
2	C _t WM C', R _n , C
3	F MD _{out} , R=B IR _i
4	R _{out} ^B , R' _{out} ⁿ , SelectA Add R _i , En
4	A 5 B, , 6 n d

Figure 7.9. Control sequence for the instruction. Add R4,R5,R6,
for the three-bus organization in Figure 7.8.

Hardwired Control

- To execute instructions, the processor must have some means of generating the control signals needed in the proper sequence.
- Two categories:
- **hardwired control and microprogrammed control**
- Hardwired system can operate at high speed; but with little flexibility.

- Consider the sequence of control signals given in figure 7.6
- Each step is completed in one clock period.
- A counter can be used to keep track of control steps.
- Each state or count of this counter corresponds to one control step.
- The required control signals are determined by the following information:
 - Contents of the control step counter.
 - Contents of the instruction register.
 - Contents of condition code flags
 - External input signals such as MFC and interrupt requests

Control Unit Organization

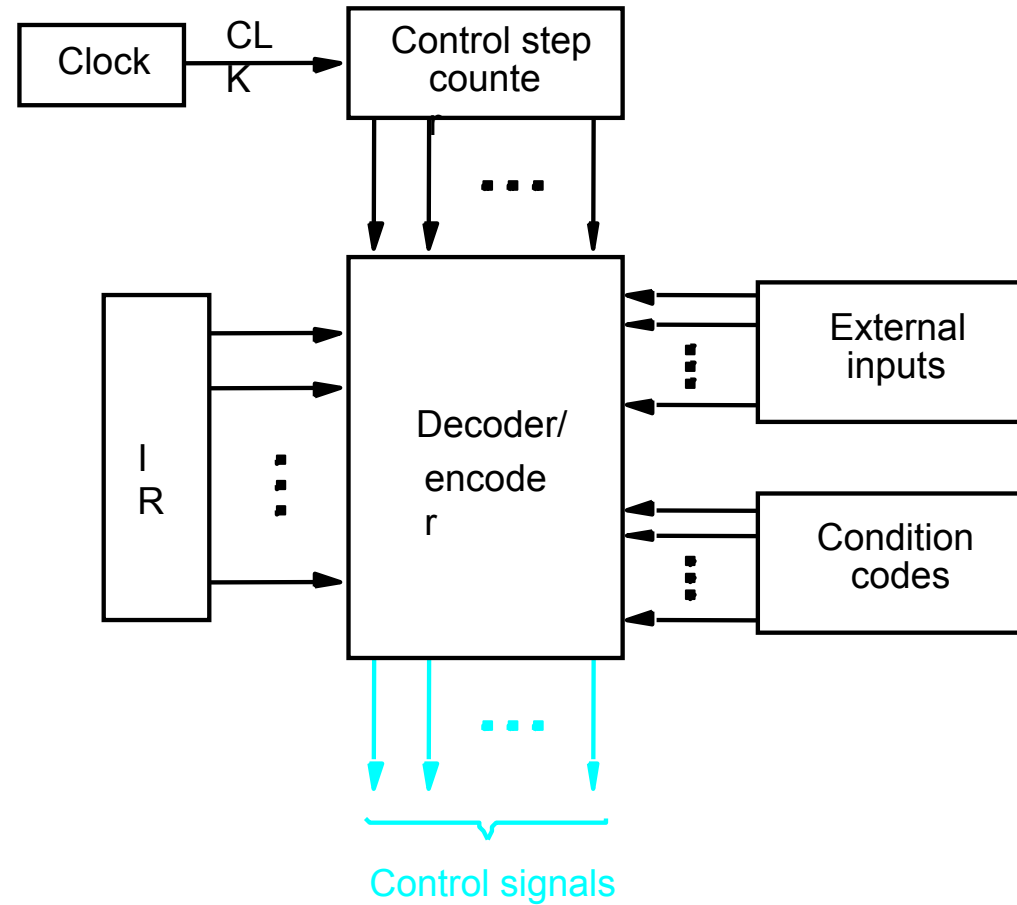


Figure 7.10. Control unit organization.

Detailed Block Description

- Decoder/encoder block is a combinational-circuit that generates required control-outputs depending on state of all its inputs.
- Step-decoder provides a separate signal line for each step in the control sequence.
- Similarly, output of instruction-decoder consists of a separate line for each machine instruction.
- For any instruction loaded in IR, one of the output-lines INS_1 through INS_m is set to 1, and all other lines are set to 0.
- The input signals to encoder-block are combined to generate the individual control-signals Y_{in} , PC_{out} , Add, End and so on.

Generating Z_{in}

- $Z_{in} = T_1 + T_6 \cdot \text{ADD} + T_4 \cdot \text{BR} + \dots$

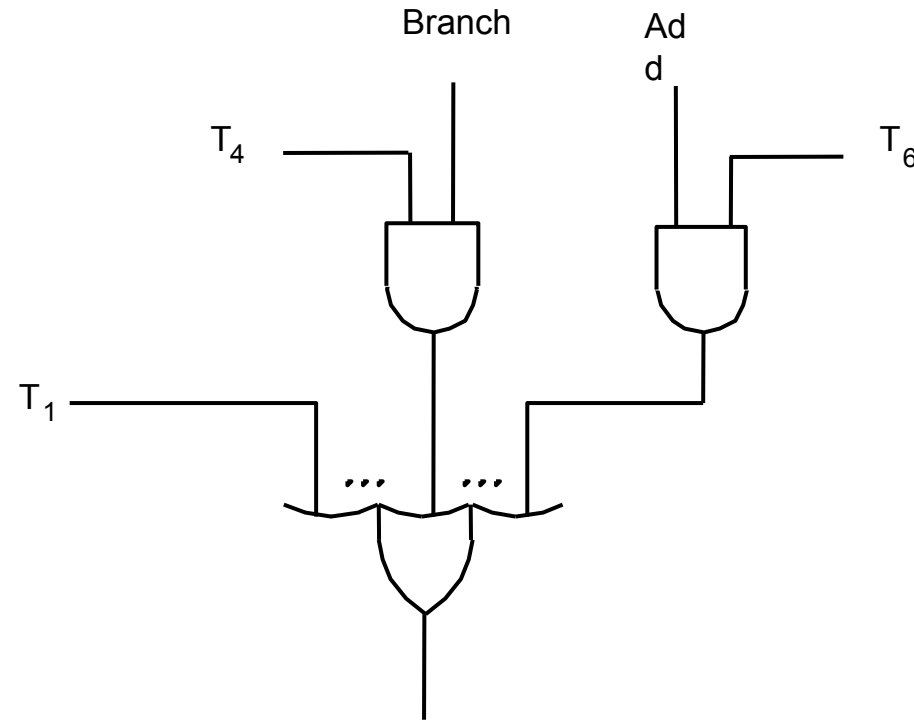


Figure 7.12. Generation of the Z_{in} control signal for the processor in Figure 7.1.

- For example, $Zin = T1 + T6.ADD + T4.BR$;
 - This signal is asserted during time-slot T1 for all instructions,
 - during T6 for an Add instruction
 - during T4 for unconditional branch instruction
- When RUN=1, counter is incremented by 1 at the end of every clock cycle. When RUN=0, counter stops counting.
- Sequence of operations carried out by this machine is determined by wiring of logic elements, hence the name “hardwired”.